

+

New

AI Chat

Home

Skills

Clap

Workflows

Drive

...

More

Algorithmic trading engine audit request

Engine_1.py
150.53 KB

Engine_1_trade...
10.59 KB

mt5_broker.py
8.19 KB

System Context & Mission Brief

You are auditing a Python-based algorithmic trading engine ('Engine_1.py') and its MT5 bridge ('mt5_broker.py'). Recently, the system transitioned from a 0.5% risk paper-trading state (which was profitable) to a 1.0% risk state integrated with MT5. Immediately after this change, profitability collapsed (Win/Loss ratio dropped to 0.39:1).

We recently discovered and patched a critical "synthetic slippage" bug where a Coinglass price scraper was feeding rounded, low-precision prices (e.g., 0.85 instead of 0.84362) which triggered stop-losses prematurely and magnified losses at the 1.0% risk level. We applied a 4-layer fix (grace buffers, exit price clamping, source tagging, and risk reduction).

Your mission is to find the "remaining" hidden issues that are destroying profitability.

Audit Objectives

Please conduct a deep-dive audit on the following three areas. Do not focus on basic Python syntax; look for architectural flaws, race conditions, and logic leaks.

1. MT5 Broker Execution ('mt5_broker.py')

Now that the internal engine is clamping its 'exit_price' to exact mathematical SL/TP levels, we need to ensure the MT5 execution matches reality.

- * **Slippage & Latency** Are there blocking calls or latency bottlenecks in 'mt5_broker.py' that cause the actual MT5 fill price to diverge significantly from the Engine's internal trigger price?
- * **Order Type Flaws** Are we using market orders when we should be using limit orders for TP? Look at how the engine transmits exits to MT5.
- * **Thread Safety** The Engine runs an 'asyncio' loop but MT5 is synchronous. Are we handling the thread boundary correctly, or are order executions blocking the main price update loop?

2. Strategy Signal Logic ('Engine_1.py' lines ~900-1000)

We recently altered the trailing stop activation ('trail_act') from 0.5R to 1.0R.

- * **Entry Logic Review** Review the 'predict_proba' threshold (currently '0.40') and the 4H trend gate. Are these filters too tight/loose given the recent market conditions?
- * **Trailing Stop Mechanics** Look at the 'check_exits' method. Does waiting for 1.0R to activate a break-even trail leave us too exposed to mean-reversion chop?
- * **Cooldown Logic** We implemented a 30-min SL cooldown and 1-hour TP cooldown. Is there a flaw here that prevents the engine from catching immediate re-entries during massive liquidations?

3. Data Feed Precision & Speed

- * Review the 'SnapshotStore' and 'BinanceFootprintFeed'. We attempted to lock Binance symbols to high-precision websocket data, blocking Coinglass rounded prices.
- * Is there any path where Coinglass data can still override or pollute the execution prices used in 'check_exits'?

Deliverables

Please provide:

- * **A Root Cause Analysis** Identify any structural flaws in the MT5 integration or entry logic that contribute to a low win rate.
- * **Actionable Code Patches** Provide exact Python snippets to fix the identified bottlenecks. Focus on execution speed and signal fidelity.
- * **Strategy Tuning Recommendations** Should we revert trailing stops, adjust the 'predict_proba' threshold, or modify the trend gate?

Executive finding

The synthetic-slippage fix was necessary, but the remaining profitability killers are likely these:

1. **MT5 is not actually synchronized with the engine's internal exits.** Entries may be sent, but trailing stop changes, emergency exits, timeout exits, and Internal SL/TP exits are not transmitted/reconciled with MT5.
2. **MT5 quote basis mismatch is huge on some symbols.** Your log shows e.g. DOTUSD1 engine entry 0.84 while MT5 entry is 0.815-0.837. That basis is larger than the stop distance. Those symbols should be skipped or priced from MT5.
3. **The liquidation model direction logic is probably wrong.** trigger_type is computed but then ignored when mapping Breakout/Reversal into LONG/SHORT. That can invert half the strategy.
4. **Risk is mismatched.** Internal engine uses 0.005 risk, MT5 bridge uses 0.01. If live, MT5 is risking 2x what the engine thinks.
5. **Binance price feed is not websocket/high-speed.** It is REST kline polling every ~2s. Also Coinglass rounded prices still drive commodities, so the old precision issue remains for XAU/XAG/CL/NATGAS.
6. **Trailing stop activation is hardcoded to 1.0R, ignoring trail_act.** For liquidation mean-reversion, that likely leaves trades exposed too long.
7. **Cooldown is strategy-inconsistent.** Alpha uses cooldown; Liquidation ignores it. If made global naively, it may miss multi-candle liquidation cascades.

Below are concrete patches.

1. Root cause analysis

A. MT5 execution / broker integration

A1. dry_run=True is still hardcoded

In Engine1TradeTracker.__init__:

```
self.mt5_broker = MT5Broker(dry_run=True, account_size=initial_capital, risk_pct=0.01)
```

So in the pasted code, MT5 orders are **not live**. If you changed this elsewhere, ignore this point. But as shown, the engine logs MT5-like values while not sending orders.

A2. Internal exits are not sent to MT5

execute_trade() sends entry with initial SL/TP, but check_exits() only updates local JSON state. It does **not**:

- close MT5 position on internal SL/TP/TIMEOUT,
- close MT5 position on emergency halt,
- modify MT5 SL when the engine trails to break-even,
- reconcile if broker SL/TP hit before the engine notices.

This is a major live-trading mismatch. Internal clamping to exact SL/TP makes paper results cleaner, but

real MT5 fills happen on broker bid/ask with spread/slippage.

Fix: store MT5 position ticket, add `close_position()` and `modify_sltp()` methods, and call them from `check_exits()` / emergency halt.

A3. MT5 basis mismatch can exceed your stop distance

From your log:

```
"symbol": "DOTUSD1",
"entry_price": 0.84,
"mt5_entry": 0.827
```

That is roughly a **1.55% basis difference**. Your stop distance is around $0.0036 / 0.84 = 0.43\%$. The broker quote is more than 3x the stop distance away from your signal market.

Even if you proportionally translate SL/TP, the strategy signal was generated on one market and executed on a materially different one. This will destroy edge.

Fix: reject MT5 trades when MT5-vs-engine basis exceeds a fraction of stop distance.

A4. Risk mismatch: engine 0.5%, MT5 1.0%

Internal engine:

```
risk_capital = max(0.0, self.current_capital) * 0.005 * risk_mult
```

MT5 bridge:

```
MT5Broker(... risk_pct=0.01)
```

So the paper tracker thinks risk is 0.5%, while MT5 lot sizing risks 1.0%. That makes live drawdown twice what the engine expects.

Fix: define one global `ENGINE_RISK_PCT` and use it everywhere.

A5. Lot sizing should use `order_calc_profit()`, not only tick value

For CFDs/crypto CFDs, `trade_tick_value` can be broker-specific and unreliable. Safer:

```
loss_per_lot = abs(mt5.order_calc_profit(order_type, symbol, 1.0, entry, sl))
lot = risk_usd / loss_per_lot
```

Also, current code does this:

```
lot = max(min_lot, min(lot, max_lot))
```

If `raw_lot < min_lot`, this **forces minimum lot** and can over-risk the account. Better to skip the trade.

2. Strategy signal logic issues

B1. Liquidation direction mapping likely ignores the liquidation side

You compute:

```
if liq_long_z_200 >= 3.0:
    trigger_type = 1
elif liq_short_z_200 >= 3.0:
    trigger_type = -1
```

But then direction is chosen as:

```
if prob_reversal > prob_breakout:
    direction = 1
elif prob_breakout > prob_reversal:
    direction = -1
```

That means:

- every reversal becomes LONG,
- every breakout becomes SHORT,

regardless of whether the spike was long-liquidation or short-liquidation.

Correct pattern logic should usually be:

```
long liquidation spike = downside flush
reversal => LONG
breakout => SHORT

short liquidation spike = upside squeeze
reversal => SHORT
breakout => LONG
```

This is probably the biggest remaining strategy-side bug

B2. `predict_proba >= 0.40` is too loose for a 3-class model

For 3 classes, random baseline is about 0.333. A threshold of 0.40 is not very selective.

Current code also does not require:

- edge over Hold class,
- edge over opposite directional class,
- 2-of-3 model agreement.

This can turn weak liquidation events into frequent low-quality trades.

B3. 4H trend gate is too blunt

Current gate:

```
if (direction == 1 and trend_4h == -1) or (direction == -1 and trend_4h == 1):
    return
```

For breakout/follow-through trades, this is reasonable.

For liquidation reversal trades, it can block the best exhaustion trades. Better:

- apply strict 4H gate to breakout trades,
- allow reversal trades against trend only with higher confidence or extreme liquidation z-score.

B4. trail_act is ignored

In check_exits():

```
if cur_r >= 1.0:  
    ns = entry_price
```

This ignores:

```
trail_act = trade.get('trail_act')
```

So changing trail_act from 0.5 to 1.0 elsewhere does not work unless you hardcoded this line. For liquidation reversals, waiting until 1R before break-even likely leaves too much exposure to chop.

B5. Cooldown is not applied consistently

Alpha checks:

```
cooldown_until = trade_tracker.reentry_cooldown_until.get(symbol, 0)
```

Liquidation does not check cooldown before calling trigger_entry().

Also, your cooldown is symbol-only, not strategy-aware. A liquidation TP should not necessarily block Alpha, and vice versa.

3. Data feed precision / speed issues

C1. Binance feed is REST kline polling, not high-speed websocket

BinanceFootprintFeed calls:

```
https://fapi.binance.com/fapi/v1/klines
```

every ~2 seconds.

That is fine for footprint approximation, but it is too slow for execution monitoring. For exits, use websocket trade/mark/book data or broker ticks.

C2. Coinglass can still drive commodity execution prices

This logic blocks Coinglass price for crypto:

```
if source == "coinglass" and is_binance_symbol:  
    continue
```

But for:

```
XAUUSD, XAGUSD, CLUSD, NATGASUSD
```

Coinglass price is still accepted. Therefore the old rounded-price issue remains for commodities.

Either:

1. feed commodities from MTS ticks, or
2. disable commodity trading until MTS tick feed is fixed.

C3. ts_ns freshness hides stale prices

AssetSnapshot.ts_ns updates on any field update. So a Coinglass indicator update can make a stale Binance price look fresh on the dashboard.

You need separate timestamps:

```
price_ts_ns  
indicator_ts_ns  
price_source
```

4. Actionable code patches

Patch 1 — unify risk and live/dry-run config

Add near global config:

```
ENGINE_RISK_PCT = float(os.environ.get("ENGINE_RISK_PCT", "0.005")) # start with 0.5%  
MTS_LIVE = os.environ.get("MTS_LIVE", "0") == "1"
```

In EngineTradeTracker.__init__ replace:

```
self.mts_broker = MTSBroker(dry_run=True, account_size=initial_capital, risk_pct=0.01)
```

with:

```
self.mts_broker = MTSBroker(  
    dry_run=not MTS_LIVE,  
    account_size=initial_capital,  
    risk_pct=ENGINE_RISK_PCT,  
)
```

In trigger_entry() replace:

```
risk_capital = max(0.0, self.current_capital) * 0.005 * risk_mult
```

with:

```
risk_capital = max(0.0, self.current_capital) * ENGINE_RISK_PCT * risk_mult
```

Patch 2 — fix Liquidation direction mapping and confidence gates

In LiveLiquidationPredictor._run_inference(), replace the block from:

```
prob_breakout = ens_probs[0, 0]  
prob_reversal = ens_probs[0, 2]  
  
direction = 0  
confidence = 0.0  
if prob_reversal > prob_breakout and prob_reversal > 0.40:
```

```

        direction = 1
        confidence = prob_reversal
    elif prob_breakout > prob_reversal and prob_breakout >= 0.40:
        direction = -1
        confidence = prob_breakout

```

through the current 4H trend gate with:

```

prob_breakout = float(ens_probs[0, 0])
prob_hold = float(ens_probs[0, 1])
prob_reversal = float(ens_probs[0, 2])

per_model_probs = [lgb_probs, xgb_probs, cb_probs]

BREAKOUT_CLASS = 0
REVERSAL_CLASS = 2

def class_votes(class_idx: int) -> int:
    return sum(int(np.argmax(p[0]) == class_idx) for p in per_model_probs)

# More selective than 0.40. Tune with OOS, but do not run 0.40 live.
MIN_REVERSAL_CONF = 0.50
MIN_BREAKOUT_CONF = 0.52
MIN_EDGE_VS_HOLD = 0.07
MIN_EDGE_VS_OTHER_DIRECTION = 0.04
MIN_MODEL_VOTES = 2

pattern = None
confidence = 0.0

if (
    prob_reversal >= MIN_REVERSAL_CONF
    and prob_reversal - prob_hold >= MIN_EDGE_VS_HOLD
    and prob_reversal - prob_breakout >= MIN_EDGE_VS_OTHER_DIRECTION
    and class_votes(REVERSAL_CLASS) >= MIN_MODEL_VOTES
):
    pattern = "reversal"
    confidence = prob_reversal

elif (
    prob_breakout >= MIN_BREAKOUT_CONF
    and prob_breakout - prob_hold >= MIN_EDGE_VS_HOLD
    and prob_breakout - prob_reversal >= MIN_EDGE_VS_OTHER_DIRECTION
    and class_votes(BREAKOUT_CLASS) >= MIN_MODEL_VOTES
):
    pattern = "breakout"
    confidence = prob_breakout

else:
    return

# trigger_type convention:
# +1 = long-liquidation spike / downside flush
# -1 = short-liquidation spike / upside squeeze
if pattern == "reversal":
    direction = 1 if trigger_type == 1 else -1
else: # breakout / continuation
    direction = -1 if trigger_type == 1 else 1

# Trend gate:
# - strict for breakout continuation
# - softer for liquidation reversal
opposing_4h_trend = (
    (direction == 1 and trend_4h == -1)
    or (direction == -1 and trend_4h == 1)
)

max_liq_z = max(float(liq_long_z_200), float(liq_short_z_200))

if pattern == "breakout" and opposing_4h_trend:
    return

# Allow strong exhaustion reversals against trend, but require stronger evidence.
if pattern == "reversal" and opposing_4h_trend:
    if confidence < 0.58 and max_liq_z < 4.0:
        return

```

Then when calling `trigger_entry()`, change:

```
trail_act=1.0
```

to:

```
trail_act=0.60 if pattern == "reversal" else 0.90
```

Patch 3 — make trailing stop actually use `trail_act`

In `Engine1TradeTracker.check_exits()`, replace this part:

```

if sl_dist and trail_act and current_atr > 0:
    if direction == 1:
        cur_r = (current_price - entry_price) / sl_dist
        if cur_r >= 1.0:
            ns = entry_price
            if ns > sl:
                trade['sl'] = ns
                sl = ns
    else:
        cur_r = (entry_price - current_price) / sl_dist
        if cur_r >= 1.0:
            ns = entry_price
            if ns < sl:
                trade['sl'] = ns
                sl = ns

```

with:

```

activation_r = float(trade.get("trail_act", 1.0) or 0.0)

if sl_dist and activation_r > 0.0 and current_atr > 0:
    if direction == 1:
        cur_r = (current_price - entry_price) / sl_dist
        if cur_r >= activation_r:
            ns = entry_price # break-even stop
            if ns > sl:
                trade['sl'] = ns
                sl = ns

    # If live MTS, push SL modification to broker.
    if trade.get("mt5_ticket") and not self.mt5_broker.dry_run:
        mt5_sl = self._translate_to_mt5_price(trade, sl)
        mt5_tp = self._translate_to_mt5_price(trade, trade["tp"])
        self.broker_submit(
            self.mt5_broker.modify_sltp,
            trade["mt5_symbol"],
            trade["mt5_ticket"],
            mt5_sl,
            mt5_tp

```

```

        mt5_tp,
    )
    else:
        cur_r = (entry_price - current_price) / sl_dist
        if cur_r >= activation_r:
            ns = entry_price # break-even stop
            if ns < sl:
                trade["sl"] = ns
                sl = ns

            if trade.get("mt5_ticket") and not self.mt5_broker.dry_run:
                mt5_sl = self._translate_to_mt5_price(trade, sl)
                mt5_tp = self._translate_to_mt5_price(trade, trade["tp"])
                self.broker.submit(
                    self.mt5_broker.modify_sltp,
                    trade["mt5_symbol"],
                    trade["mt5_ticket"],
                    mt5_sl,
                    mt5_tp,
                )
    )

```

Copy

Add these helper methods to Engine1TradeTracker:

```

def _translate_to_mt5_price(self, trade: dict, engine_price: float) -> float:
    """
    Translate an Engine/Binance-side price to MT5 price space using proportional distance.
    """
    if not trade.get("mt5_entry") or not trade.get("entry_price"):
        return engine_price
    return float(trade["mt5_entry"]) * (float(engine_price) / float(trade["entry_price"]))

def _broker_submit(self, fn, *args, **kwargs) -> None:
    """
    Fire-and-forget MT5 action so the asyncio price loop is not blocked.
    """
    try:
        self.broker_executor.submit(fn, *args, **kwargs)
    except Exception as exc:
        print(f"[MT5] Failed to submit broker action: {exc}")

```

Copy



In Engine1TradeTracker.__init__, add:

```

from concurrent.futures import ThreadPoolExecutor
self.broker_executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="mt5-broker")

```

Copy

Patch 4 — strategy-aware cooldown

Add to Engine1TradeTracker:

```

def _cooldown_key(self, strategy: str, symbol: str) -> str:
    return f"{strategy}:{symbol}"

def _cooldown_secs_after_close(self, strategy: str, reason: str) -> int:
    if strategy == "ML_Liquidation_Runner":
        # Liquidation cascades can cluster. Do not block fresh TP re-entries for 1 hour.
        if reason == "TP":
            return 0
        if reason in ("SL", "BE", "TRAIL"):
            return 15 * 60
        return 15 * 60

    if strategy == "AlphaSqueezer_V17":
        if reason == "TP":
            return self.REENTRY_COOLDOWN_TP_SECS
            return self.REENTRY_COOLDOWN_SL_SECS

    return 15 * 60

```

Copy



In trigger_entry(), near the top after emergency halt check:

```

cool_key = self._cooldown_key(strategy, symbol)
cooldown_until = self.reentry_cooldown_until.get(cool_key, 0.0)

if time.time() < cooldown_until:
    print(
        f"[RiskGovernor] Entry blocked by cooldown. "
        f"Symbol={symbol} Strategy={strategy} "
        f"Remaining={cooldown_until - time.time():.0f}s"
    )
    return

```

Copy

In check_exits(), replace:

```

cooldown_secs = self.REENTRY_COOLDOWN_TP_SECS if reason == "TP" else
self.REENTRY_COOLDOWN_SL_SECS
self.reentry_cooldown_until[symbol] = time.time() + cooldown_secs

```

Copy



with:

```

closed_strategy = trade.get("strategy", "")
cooldown_secs = self._cooldown_secs_after_close(closed_strategy, reason)

if cooldown_secs > 0:
    self.reentry_cooldown_until[
        self._cooldown_key(closed_strategy, symbol)
    ] = time.time() + cooldown_secs

```

Copy

Also update the Alpha-specific cooldown lookup inside LiveStrategyPredictor:

```

cooldown_until = max(
    trade_tracker.reentry_cooldown_until.get(f"AlphaSqueezer_V17:{symbol}", 0.0),
    trade_tracker.reentry_cooldown_until.get(symbol, 0.0), # backward compatibility
)

```

Copy

Patch 5 — MT5 broker basis guard, ticket storage, close/modify methods

In mt5_broker.py, add threading:

```

import threading

```

Copy

Update MT5Broker.__init__:

```

def __init__(
    self,
    dry_run=True,
    account_size=5000.0,
    risk_governor=None,

```

Copy



```

risk_pct=0.005,
symbol_map=None,
max_abs_basis_pct=0.005,
):
    self.dry_run = dry_run
    self.account_size = account_size
    self.risk_pct = risk_pct
    self.risk_usd = self.account_size * self.risk_pct
    self.connected = False
    self._lock = threading.RLock()
    self._symbol_cache = {}
    self.symbol_map = symbol_map or {}
    self.max_abs_basis_pct = max_abs_basis_pct

```

Replace `_map_symbol()` with a safer cached version:

```

def _map_symbol(self, binance_symbol):
    if binance_symbol in self.symbol_map:
        return self.symbol_map[binance_symbol]

    if binance_symbol in self._symbol_cache:
        return self._symbol_cache[binance_symbol]

    base_sym = binance_symbol.replace("USDT", "USD")

    if not self.connected:
        return base_sym

    syms = mt5.symbols_get(groups=f"*{base_sym}*")
    if not syms:
        self._symbol_cache[binance_symbol] = base_sym
        return base_sym

    # Prefer exact, visible, trade-enabled symbol.
    candidates = list(syms)

    def score(s):
        exact = 1 if s.name == base_sym else 0
        visible = 1 if getattr(s, "visible", False) else 0
        trade_allowed = 1 if getattr(s, "trade_mode", 0) != mt5.SYMBOL_TRADE_MODE_DISABLED else 0
        return (exact, visible, trade_allowed)

    chosen = sorted(candidates, key=score, reverse=True)[0].name
    self._symbol_cache[binance_symbol] = chosen
    return chosen

```

Add helpers:

```

def _normalize_price(self, sym_info, price: float) -> float:
    return round(float(price), int(sym_info.digits))

def _normalize_lot(self, sym_info, raw_lot: float) -> float | None:
    min_lot = float(sym_info.volume_min)
    max_lot = float(sym_info.volume_max)
    step = float(sym_info.volume_step)

    if raw_lot < min_lot:
        return None # Do not force min lot <= 0.01 per-risk.

    lot = math.floor(raw_lot / step) * step
    lot = min(lot, max_lot)
    return round(lot, 0)

def _loss_per_lot(self, mt5_sym: str, order_type: int, entry: float, sl: float, sym_info) -> float:
    calc = mt5.order_calc_profit(order_type, mt5_sym, 1.0, entry, sl)
    if calc is not None and calc < 0:
        return abs(float(calc))

    # Fallback.
    tick_size = float(sym_info.trade_tick_size)
    tick_value = float(sym_info.trade_tick_value)
    if tick_size <= 0 or tick_value <= 0:
        return 0.0
    return abs(entry - sl) / tick_size * tick_value

```

Replace `execute_trade()` with:

```

def execute_trade(self, binance_symbol, direction, bin_entry, bin_sl, bin_tp):
    if not self.connected:
        return None

    with self._lock:
        mt5_sym = self._map_symbol(binance_symbol)

        if not mt5.symbol_select(mt5_sym, True):
            print(f"[MT5 SKIP] {binance_symbol} -> {mt5_sym} not listed.")
            return None

        sym_info = mt5.symbol_info(mt5_sym)
        if sym_info is None:
            print(f"[MT5 SKIP] {mt5_sym}: symbol_info unavailable.")
            return None

        tick = mt5.symbol_info_tick(mt5_sym)
        if tick is None or (tick.bid == 0.0 and tick.ask == 0.0):
            print(f"[MT5 SKIP] {mt5_sym}: no live tick.")
            return None

        order_type = mt5.ORDER_TYPE_BUY if direction == 1 else mt5.ORDER_TYPE_SELL
        mt5_entry = float(tick.ask if direction == 1 else tick.bid)

        if bin_entry <= 0:
            return None

        sl_pct_dist = abs(bin_entry - bin_sl) / bin_entry
        tp_pct_dist = abs(bin_entry - bin_tp) / bin_entry
        basis_pct = abs(mt5_entry - bin_entry) / bin_entry

        # Reject if broker quote is too far from signal market.
        # Allows small stable basis, rejects cases like 007 0.84 vs 0.827.
        max_basis_allowed = min(
            self.max_abs_basis_pct,
            max(0.0015, 0.50 * sl_pct_dist),
        )

        if basis_pct > max_basis_allowed:
            print(
                f"[MT5 SKIP] {binance_symbol} -> {mt5_sym}: basis too large. "
                f"Engine={bin_entry:.8f}, MT5={mt5_entry:.8f}, "
                f"basis={basis_pct*100:.3f}%, allowed={max_basis_allowed*100:.3f}%"
            )
            return None

        if direction == 1:
            mt5_sl = mt5_entry * (1.0 - sl_pct_dist)
            mt5_tp = mt5_entry * (1.0 + tp_pct_dist)
        else:
            mt5_sl = mt5_entry * (1.0 + sl_pct_dist)
            mt5_tp = mt5_entry * (1.0 - tp_pct_dist)

```

```

mt5_sl = self._normalize_price(sym_info, mt5_sl)
mt5_tp = self._normalize_price(sym_info, mt5_tp)
mt5_entry = self._normalize_price(sym_info, mt5_entry)

# Broker minimum stop/freeze-level guard.
point = float(sym_info.point or sym_info.trade_tick_size or 0.0)
min_stop_dist = max(
    float(getattr(sym_info, "trade_stops_level", 0)) * point,
    float(getattr(sym_info, "trade_freeze_level", 0)) * point,
)

if min_stop_dist > 0:
    if abs(mt5_entry - mt5_sl) < min_stop_dist or abs(mt5_entry - mt5_tp) <
min_stop_dist:
        print(
            f"[M5 SKIP] {mt5_sym}: SL/TP inside broker minimum stop distance. "
            f"min={min_stop_dist}, entry={mt5_entry}, sl={mt5_sl}, tp={mt5_tp}"
        )
        return None

acc_info = mt5.account_info()
current_balance = acc_info.balance if acc_info is not None else self.account_size
risk_usd = current_balance * self.risk_pct

loss_per_lot = self._loss_per_lot(mt5_sym, order_type, mt5_entry, mt5_sl, sym_info)
if loss_per_lot <= 0:
    print(f"[M5 SKIP] {mt5_sym}: can't compute loss_per_lot.")
    return None

raw_lot = risk_usd / loss_per_lot
lot = self._normalize_lot(sym_info, raw_lot)

if lot is None:
    print(
        f"[M5 SKIP] {mt5_sym}: min lot would exceed risk. "
        f"raw_lot={raw_lot:.6f}, min_lot={sym_info.volume_min}"
    )
    return None

# Deviation is in broker points, not percent.
max_slip_pct = min(0.0003, max(0.00005, 0.05 * sl_pct_dist))
deviation_points = max(20, int((mt5_entry * max_slip_pct) / point)) if point > 0
else 20

if self.dry_run:
    print(f"[M5 DRY RUN] {mt5_sym} | {'LONG' if direction == 1 else 'SHORT'}")
    print(f"  Engine Entry: {bin_entry:.8f} | M5 Entry: {mt5_entry:.8f}")
    print(f"  Basis: {basis_pct*100:.3f}% | Allowed: {max_basis_allowed*100:.3f}%")
    print(f"  M5 SL: {mt5_sl:.8f} | M5 TP: {mt5_tp:.8f}")
    print(f"  Lot: {lot:.4f} | Risk: ${risk_usd:.2f}")
    return {
        "mt5_symbol": mt5_sym,
        "mt5_ticket": None,
        "mt5_entry": mt5_entry,
        "mt5_sl": mt5_sl,
        "mt5_tp": mt5_tp,
        "lot": lot,
        "basis_pct": basis_pct,
    }

request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": mt5_sym,
    "volume": float(lot),
    "type": order_type,
    "price": mt5_entry,
    "sl": mt5_sl,
    "tp": mt5_tp,
    "deviation": deviation_points,
    "magic": 234000,
    "comment": "Engine1",
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": mt5.ORDER_FILLING_IOC,
}

check = mt5.order_check(request)
if check is not None and check.retcode not in (0, mt5.TRADE_RETCODE_DONE):
    print(f"[M5 SKIP] order_check failed: retcode={check.retcode}, comment={
(check.comment)}")
    return None

result = mt5.order_send(request)
if result is None or result.retcode != mt5.TRADE_RETCODE_DONE:
    code = result.retcode if result else "None"
    comment = result.comment if result else "None"
    print(f"[M5] Order failed. Code={code}, Comment={comment}")
    return None

# Find latest position ticket for this symbol/magic.
position_ticket = None
positions = mt5.positions_get(symbol=mt5_sym)
if positions:
    mine = [p for p in positions if getattr(p, "magic", None) == 234000]
    candidates = mine or list(positions)
    latest = max(candidates, key=lambda p: getattr(p, "time_msc", getattr(p, "time",
0)))
    position_ticket = int(latest.ticket)

print(f"[M5 LIVE] Order sent: {mt5_sym}, ticket={position_ticket}")

return {
    "mt5_symbol": mt5_sym,
    "mt5_ticket": position_ticket,
    "mt5_order": int(result.order),
    "mt5_deal": int(result.deal),
    "mt5_entry": mt5_entry,
    "mt5_sl": mt5_sl,
    "mt5_tp": mt5_tp,
    "lot": lot,
    "basis_pct": basis_pct,
}

```

Copy

Add M5 modify/close methods:

```

def modify_sltp(self, mt5_sym: str, position_ticket: int, sl: float, tp: float) -> bool:
    if self.dry_run:
        print(f"[M5 DRY RUN] Modify SLTP {mt5_sym} ticket={position_ticket} SL={sl} TP=
{tp}")
        return True

    if not self.connected or not position_ticket:
        return False

    with self._lock:
        sym_info = mt5.symbol_info(mt5_sym)
        if sym_info is None:
            return False

        sl = self._normalize_price(sym_info, sl)
        tp = self._normalize_price(sym_info, tp)

        request = {
            "action": mt5.TRADE_ACTION_SLTP,
            "symbol": mt5_sym,
            "position": int(position_ticket),
            "sl": sl,
            "tp": tp,

```

```

        "magic": 234000,
        "comment": "Engine1 SLTP modify",
    }

    result = mt5.order_send(request)
    if result is None or result.retcode != mt5.TRADE_RETCODE_DONE:
        code = result.retcode if result else "None"
        comment = result.comment if result else "None"
        print(f"[MT5] SLTP modify failed. Code={code}, Comment={comment}")
        return False

    print(f"[MT5] SLTP modified: {mt5_sym} ticket={position_ticket} SL={sl} TP={tp}")
    return True

def close_position(self, position_ticket: int, reason: str = "ENGINE_EXIT") -> bool:
    if self.dry_run:
        print(f"[MT5 DRY RUN] Close position ticket={position_ticket}, reason={reason}")
        return True

    if not self.connected or not position_ticket:
        return False

    with self._lock:
        positions = mt5.positions_get(ticket=position_ticket)
        if not positions:
            # Already closed by broker SL/TP.
            return True

        pos = positions[0]
        tick = mt5.symbol_info_tick(pos.symbol)
        if tick is None:
            return False

        if pos.type == mt5.POSITION_TYPE_BUY:
            close_type = mt5.ORDER_TYPE_SELL
            price = tick.bid
        else:
            close_type = mt5.ORDER_TYPE_BUY
            price = tick.ask

        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": pos.symbol,
            "position": int(position_ticket),
            "volume": float(pos.volume),
            "type": close_type,
            "price": price,
            "deviation": 50,
            "magic": 234000,
            "comment": f"Engine1 close {reason}",
            "type_time": mt5.ORDER_TIME_GTC,
            "type_filling": mt5.ORDER_FILLING_IOC,
        }

        result = mt5.order_send(request)
        if result is None or result.retcode != mt5.TRADE_RETCODE_DONE:
            code = result.retcode if result else "None"
            comment = result.comment if result else "None"
            print(f"[MT5] Close failed. ticket={position_ticket}, Code={code}, Comment={comment}")
            return False

        print(f"[MT5] Closed position ticket={position_ticket}, reason={reason}")
        return True

```

Then in `Engine1TradeTracker.trigger_entry()`, create the trade only after MT5 accepts it in live mode:

```

trade = {
    "trade_id": trade_id,
    "symbol": symbol,
    "strategy": strategy,
    "direction": direction,
    "entry_price": entry_price,
    "entry_time": time.strftime("%Y-%m-%d %H:%M:%S"),
    "entry_timestamp": time.time(),
    "sl": sl,
    "tp": tp,
    "units": units,
    "live_pnl_pct": 0.0,
    "live_pnl_usd": 0.0,
    "atr": atr,
    "macro": macro,
    "vol_regime": vol_regime,
    "sl_dist": stop_dist,
    "trail_act": trail_act,
    "trail_buf": 0.5,
}

mt5_res = self.mt5_broker.execute_trade(symbol, direction, entry_price, sl, tp)

if self.mt5_broker.connected and not self.mt5_broker.dry_run and mt5_res is None:
    print(f"[MT5] Live order rejected. Internal trade not opened: {symbol} {strategy}")
    return

if mt5_res:
    trade.update({
        "mt5_symbol": mt5_res.get("mt5_symbol"),
        "mt5_ticket": mt5_res.get("mt5_ticket"),
        "mt5_entry": mt5_res.get("mt5_entry"),
        "mt5_sl": mt5_res.get("mt5_sl"),
        "mt5_tp": mt5_res.get("mt5_tp"),
        "mt5_lot": mt5_res.get("lot"),
        "mt5_basis_pct": mt5_res.get("basis_pct"),
    })

self.active_trades[trade_id] = trade
self.save_history()

```

And when `check_exits()` closes a trade, submit a broker close before/while recording local history:

```

if trade.get("mt5_ticket") and not self.mt5_broker.dry_run:
    self._broker_submit(
        self.mt5_broker.close_position,
        trade["mt5_ticket"],
        reason,
    )

```

Patch 6 — separate price timestamp/source and block lower-quality overrides

Add to `AssetSnapshot`:

```

price_ts_ns: int = 0
price_source: str = ""

```

Add global source ranking:

```

PRICE_SOURCE_RANK = {
    "": -1,

```

```

"coinglass": 0,
"binance_rest": 1,
"binance": 1,
"binance_ws": 2,
"mt5": 3,
}

```

In `SnapshotStore.update()`, replace price-handling logic with:

```

now_ns = time.time_ns()

for k, v in patch.items():
    if not hasattr(cur, k):
        continue

    if k in ("price", "open", "high", "low", "close"):
        if k == "price":
            # Never let Coinglass override Binance symbols.
            if source == "coinglass" and is_binance_symbol:
                continue

            # Do not let slower/lower-ranked price sources override a fresh better source.
            cur_rank = PRICE_SOURCE_RANK.get(getattr(cur, "price_source", ""), -1)
            new_rank = PRICE_SOURCE_RANK.get(source, 0)
            cur_price_fresh = (now_ns - getattr(cur, "price_ts_ns", 0)) < 3_000_000_000

            if new_rank < cur_rank and cur_price_fresh:
                continue

        fv = finite_float_or_none(v)
        if fv is None:
            continue
        if k == "price" and fv <= 0.0:
            continue
        clean_patch[k] = fv
    else:
        clean_patch[k] = v

```

Then after:

```
price_updated = "price" in clean_patch
```

add:

```

if price_updated:
    clean_patch["price_ts_ns"] = now_ns
    clean_patch["price_source"] = source

```

And use `now_ns` in `dataclasses.replace`:

```
new_snap = dataclasses.replace(cur, seq=self.seq, ts_ns=now_ns, **clean_patch)
```

In `render_table()`, price freshness should use `price_ts_ns`:

```

fresh_price = (now - a.price_ts_ns) < STALE_NS
fresh_data = (now - a.ts_ns) < STALE_NS

```

Then render price using `fresh_price`, and indicators using `fresh_data`.

Patch 7 — add Binance websocket price feed

Keep REST Kline polling for footprint, but add websocket for execution price.

```

class BinanceTradePriceWebsocketFeed:
    def __init__(self, symbols: List[str], store: SnapshotStore):
        self.symbols = [
            s for s in symbols
            if s not in ("XAUUSD", "XAGUSD", "CLUSD", "WTGASUSD")
        ]
        self.store = store
        self.running = True
        self.last_heartbeat_ns = time.time_ns()
        self.last_emit_ns: Dict[str, int] = {}

    async def run(self) -> None:
        if not self.symbols:
            return

        streams = "".join(f"{s.lower()}@aggTrade" for s in self.symbols)
        url = f"wss://fstream.binance.com/stream?streams={streams}"

        while self.running:
            try:
                async with websockets.connect(
                    url,
                    ping_interval=20,
                    ping_timeout=10,
                    max_queue=4096,
                ) as ws:
                    print(f"[Binance WS] Connected trade price stream.")
                    async for raw in ws:
                        if not self.running:
                            break

                        self.last_heartbeat_ns = time.time_ns()

                        try:
                            msg = json.loads(raw)
                            data = msg.get("data", {})
                            sym = data.get("s")
                            price = finite_float_or_none(data.get("p"))

                            if sym not in self.symbols or price is None or price <= 0:
                                continue

                            # Throttle per symbol to avoid flooding the model loop.
                            now_ns = time.time_ns()
                            last_ns = self.last_emit_ns.get(sym, 0)
                            if now_ns - last_ns < 150_000_000: # 150 ms
                                continue
                            self.last_emit_ns[sym] = now_ns

                            await self.store.update(sym, source="binance_ws", price=price)
                        except Exception:
                            continue

            except Exception as e:
                print(f"[Binance WS] Disconnected/error: {e}. Reconnecting in 3s...")
                await asyncio.sleep(3.0)

```

In `main()`:

```
binance_ws = BinanceTradePriceWebsocketFeed(ALL_SYMBOLS, store)
```

Add to tasks:

```
asyncio.create_task(binance_ws.run()),
```

Copy

And include in watchdog components:

```
watchdog([tab1, tab2, binance, binance_ws], focus_lock, stop)
```

Copy

For commodities, either add an MT5 tick feed or disable them for live execution. Minimal safety gate inside predictors before trading commodities:

```
COMMODITY_SYMBOLS = {"XAUUSD", "XAGUSD", "C↓", "NATGASUSD"}

if symbol in COMMODITY_SYMBOLS and getattr(snap, "price_source", "") != "mt5":
    return snap
```

Copy

5. Strategy tuning recommendations

Liquidation strategy

Recommended live defaults after the direction bug is fixed:

```
MIN_REVERSAL_CONF = 0.50
MIN_BREAKOUT_CONF = 0.52
MIN_EDGE_VS_HOLD = 0.07
MIN_EDGE_VS_OTHER_DIRECTION = 0.04
MIN_MODEL_VOTES = 2
```

Copy

Trailing:

```
trail_act = 0.60 for reversal
trail_act = 0.90 for breakout
```

Copy

Cooldown:

```
TP: 0 to 15 min
SL: 15 to 30 min
```

Copy

Do not use a 1-hour TP cooldown for liquidation cascades. Those events can cluster.

AlphaSqueezer strategy

Keep stricter cooldown:

```
TP: 1 hour
SL: 30 min
```

Copy

Trailing at 1.0R is more defensible for trend-following Alpha than for liquidation reversal.

4H trend gate

Use this distinction:

```
Breakout:
    require alignment with 4H trend.

Reversal:
    allow counter-trend only if confidence is high or liquidation z-score is extreme.
```

Copy

Do not apply the same hard trend filter to both patterns.

Risk

Do not run 1.0% risk again until:

1. MT5 ticket reconciliation is working,
2. basis guard is rejecting bad symbols,
3. websocket/MT5 tick price feeds are active,
4. the Liquidation direction mapping is corrected,
5. internal and MT5 risk are unified.

Suggested validation path:

```
Phase 1: 0.25% risk, MT5 dry-run, compare engine-vs-MT5 basis.
Phase 2: 0.25% live on only BTC/ETH/SOL/XRP if basis acceptable.
Phase 3: 0.50% live after 100+ trades or 2 weeks.
Phase 4: 1.00% only after live broker fills match modeled R within tolerance.
```

Copy

6. Highest-priority fix order

If you only patch a few things first, do them in this order:

1. Fix Liquidation direction mapping using trigger_type.
2. Unify risk: engine and MT5 must use the same ENGINE_RISK_PCT.
3. Add MT5 basis guard and skip symbols with large broker/Binance divergence.
4. Store MT5 ticket and transmit close/modify actions.
5. Add Binance websocket price feed and separate price_ts_us.
6. Make trailing use trail_act; use 0.6R for liquidation reversals.
7. Make cooldown strategy-aware.

The synthetic slippage bug was one layer. The remaining structural issue is that your strategy signal, internal execution model, and MT5 broker reality are still not one coherent system.

📄 🍏 🗨️ ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

